

UF4

Acceso a datos con PHP

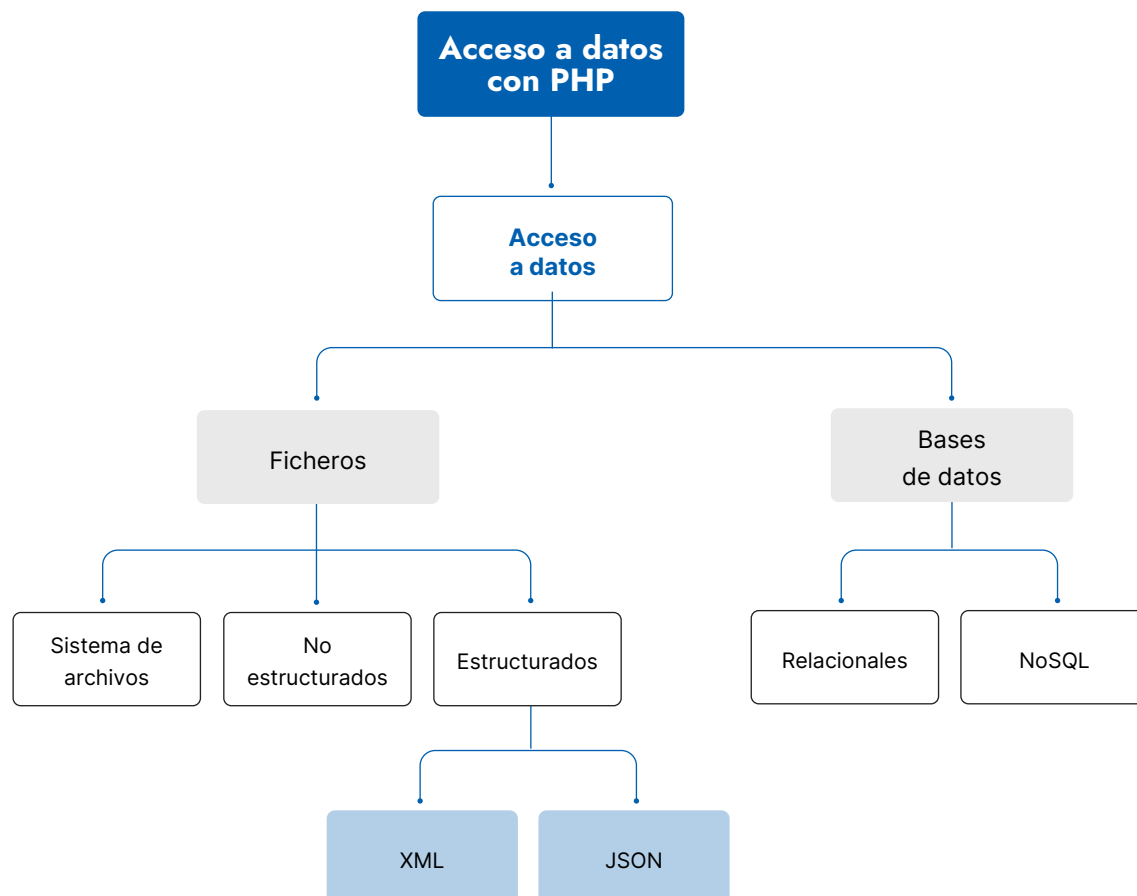
Desarrollo web en entorno servidor

ÍNDICE

Mapa conceptual	04
1. Validación de datos. Seguridad	05
1.1. Tamaño del sistema de archivos.....	05
1.2. Operaciones de comprobación sobre ficheros	06
1.3. Operaciones con directorios/carpetas.....	07
1.4. Operaciones con permisos y propietarios (en GNU/LINUX)	07
1.5. Operaciones genéricas con ficheros	08
1.6. Información sobre los ficheros	08
1.7. Lista de ficheros de un directorio	09
2. Operaciones con ficheros de texto no estructurados	13
2.1. Leer todo el contenido de un fichero	15
2.2. Leer el contenido de un fichero por partes.....	16
2.3. Escribir en un fichero	19
2.4. Leer y escribir en un fichero	20
3. Operaciones con ficheros de texto estructurados	22
3.1. PHP y XML	23
3.2. PHP y JSON.....	26
4. Operaciones con bases de datos relacionales	30
4.1. Conexión a la base de datos.....	30
4.2. Altas, bajas, modificaciones y consultas (CRUD).....	31
4.3. Seguridad	34
4.4. Transacciones.....	35
4.5. El problema de las fechas	36

5. Operaciones con bases de datos NoSQL	36
5.1. Particularidades de MongoDB	37
5.2. Conexiones y consultas básicas sobre MongoDB.....	38
5.3. Alta de documentos	39
5.4. Consulta y visualización de documentos	40
5.5. Modificación de documentos	41
5.6. Eliminar un documento.....	42

Mapa conceptual



01 Validación de datos. Seguridad

Antes de ver cómo trabajar con ficheros, es conveniente conocer la forma en que PHP nos permite acceder al propio sistema de archivos (o sistema de ficheros, **file system**) y manipularlo. Esto incluye el acceso a los directorios y carpetas, la gestión de las rutas, la consulta y la modificación de los permisos, las operaciones de copiado, subida o borrado de ficheros, o las consultas sobre el tamaño de un sistema de archivos o el espacio disponible en este, entre otras funcionalidades. Vamos a ver cómo realizar algunas de dichas operaciones, excluyendo de momento las que tienen que ver con el contenido de los ficheros, ya que estas se verán en los siguientes apartados. Para ello, enumeraremos las principales funciones relacionadas con la gestión de los archivos y veremos ejemplos de algunas de ellas.

EJEMPLO

Puedes encontrar todos los ficheros con los ejemplos descargando el archivo **DWES_U04_TEO_A01.zip**, en la sección Ejemplos para trabajar y analizar.

1.1. Tamaño del sistema de archivos

La función `disk_free_space` permite conocer el espacio disponible en el sistema de archivos o en una partición. Por otro lado, la función `disk_total_space` nos informa del espacio total del sistema de archivos o la partición de disco. En ambos casos lo hace en bytes. En Windows, por ejemplo:

```
<?php
$bytesLibres=disk_free_space("C:");
$bytesTotal=disk_total_space("C:");
$gigasLibres=round($bytesLibres/pow(1024,3),2);
$gigasTotal=round($bytesTotal/pow(1024,3),2);

echo "Tienes $gigasLibres Gigas libres de $gigasTotal";

?>
```



Figura 1.

En GNU/Linux, sustituiríamos el nombre de la unidad (`c:`) por el directorio raíz de la partición (`/`).

1.2. Operaciones de comprobación sobre ficheros

Las principales funciones para comprobar el estado de un fichero son las siguientes:

<code>file_exists(\$nombre_fichero)</code>	Devuelve true si el fichero indicado existe.
<code>is_file(\$nombre_fichero)</code>	Devuelve true si el nombre corresponde a un fichero.
<code>is_link(\$nombre_fichero)</code>	Devuelve true si el nombre corresponde a un enlace.
<code>is_executable(\$nombre_fichero)</code>	Devuelve true si el fichero es ejecutable.
<code>is_readable(\$nombre_fichero)</code>	Devuelve true si es un fichero y se puede leer.
<code>is_writable(\$nombre_fichero)</code>	Devuelve true si es un fichero y se puede escribir.
<code>is_uploaded_file(\$nombre_fichero)</code>	Devuelve true si es un fichero subido con HTTP_POST .

Esquema 1.

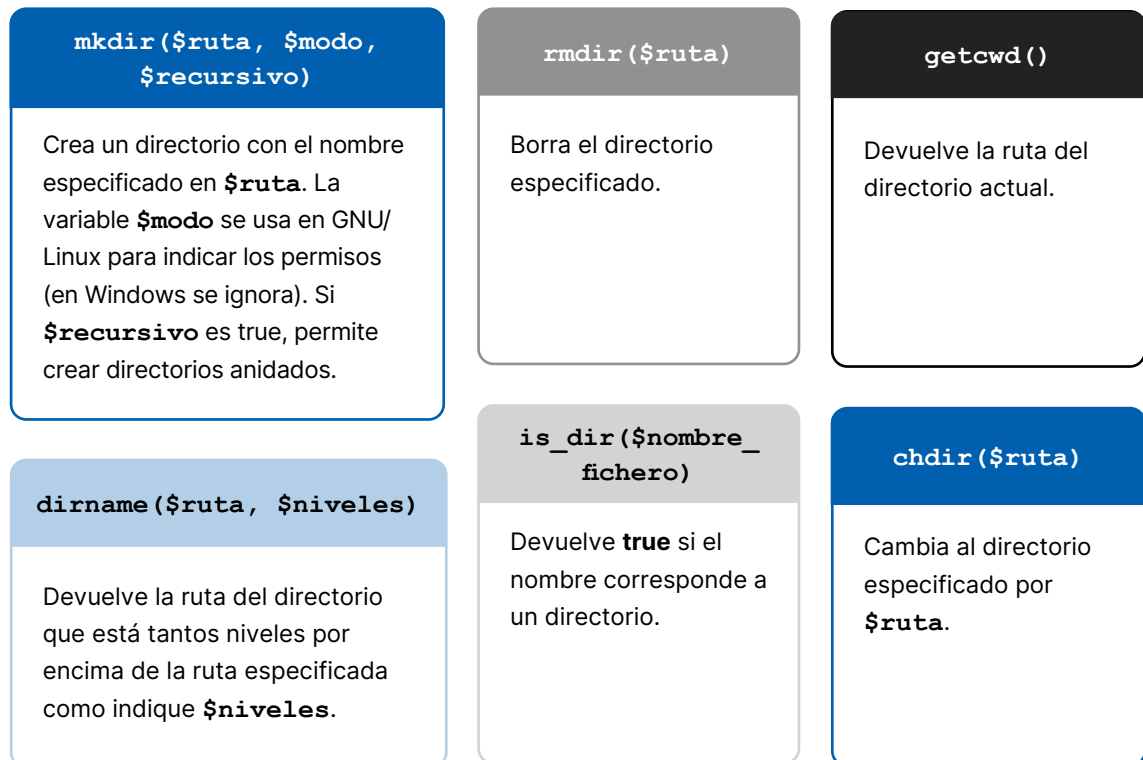
Por ejemplo, vamos a comprobar si un fichero existe y se puede modificar:

```
<?php
$nombre01="ejemplo02.php";
if(file_exists($nombre01)) {
    if(is_file($nombre01)) {
        if(is_writable($nombre01)) {
            echo "El fichero $nombre01 existe y se puede modificar";
        }
    }
    else { echo "$nombre01 no es un fichero"; }
}
else { echo "$nombre01 no existe"; }
?>
```

Ejemplo02.php

1.3. Operaciones con directorios/carpetas

Estas serían las funciones más usadas con directorios/carpetas:



Esquema 2.

Por ejemplo, el siguiente código nos dice cuál es el directorio de nivel superior al actual.

```
<?php
$actual=getcwd();
$superior=dirname($actual,1);
echo "Estás en $actual";
echo "<br/>";
echo "El directorio superior es $superior";
?>
```

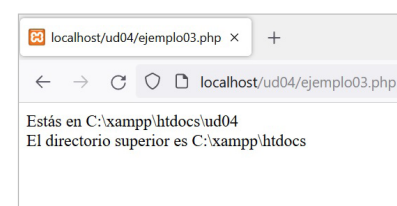


Figura 2.

1.4. Operaciones con permisos y propietarios (en GNU/LINUX)

Las siguientes funciones hacen posible trabajar con el sistema de permisos y propietarios de los ficheros, siempre que el sistema operativo lo permita.

<code>chgrp(\$nombre, \$grupo)</code>	Cambia el grupo del archivo \$nombre al grupo especificado.
<code>lchgrp(\$nombre, \$grupo)</code>	Cambia el grupo de un enlace simbólico.
<code>chmod(\$nombre, \$modo)</code>	Cambia el modo (permisos) del fichero.
<code>chown(\$nombre, \$usuario)</code>	Cambia el propietario del fichero al usuario indicado.
<code>lchown(\$nombre, \$usuario)</code>	Cambia el propietario de un enlace simbólico.
<code>filegroup(\$nombre)</code>	Devuelve el id del grupo del fichero.
<code>fileowner(\$nombre)</code>	Devuelve el id del usuario propietario del fichero.
<code>fileperms(\$nombre)</code>	Devuelve los permisos del fichero (en número).

Esquema 3.

Si dispones de un sistema operativo Linux, puedes probar el fichero adjunto **ejemplo04.php** en tu servidor.

1.5. Operaciones genéricas con ficheros

Estas serían algunas de las funciones más usadas para realizar operaciones como copiar, borrar, renombrar o mover ficheros.

<code>copy(\$origen, \$destino)</code>	Copia el fichero especificado en la ruta origen a la ruta destino.
<code>unlink(\$nombre)</code>	Borra el archivo o enlace especificado.
<code>rename(\$nombre, \$nuevo)</code>	Renombra el archivo. Si \$nuevo incluye la ruta, se puede usar para mover el archivo a la nueva ruta.
<code>move_uploaded_file(\$nombre, \$destino)</code>	Mueve un archivo subido con HTTP_POST a la ubicación especificada en \$destino .

Esquema 4.

1.6. Información sobre los ficheros

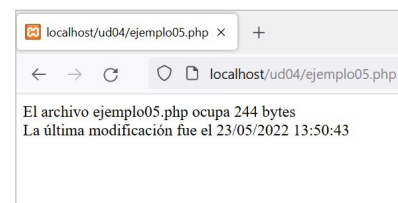
Las siguientes funciones son algunas de las más importantes a la hora de obtener información sobre el estado de un fichero.

<code>filetime(\$nombre)</code>	Devuelve el momento en que se realizó el último acceso al fichero.
<code>filemtime(\$nombre)</code>	Devuelve el momento en que se modificó el archivo por última vez.
<code>filesize(\$nombre)</code>	Devuelve el tamaño del archivo en bytes.
<code>filetype(\$nombre)</code>	Devuelve el tipo de fichero (básicamente dir o file).
<code>stat(\$nombre)</code>	Devuelve información sobre el fichero.
<code>lstat(\$nombre)</code>	Devuelve información sobre un enlace simbólico.
<code>fstat(\$puntero)</code>	Devuelve información sobre el fichero apuntado por el puntero.
<code>pathinfo(\$ruta)</code>	Devuelve información sobre el fichero y la ruta que se especifican.

Esquema 5.

Por ejemplo, hagamos que se muestre el tamaño de un archivo y la fecha de última modificación:

```
<?php
$nombre="ejemplo05.php";
$tamanyo=filesize($nombre);
$momento=filemtime($nombre);
$fecha=date("d/m/Y H:i:s",$momento);
echo "El archivo $nombre ocupa $tamanyo bytes";
echo "<br/>";
echo "La última modificación fue el $fecha";
?>
```



Ejemplo05.php

1.7. Lista de ficheros de un directorio

Para obtener el contenido de un directorio, disponemos de dos posibilidades. La primera es usar las funciones `opendir()`, `readdir()` y `closedir()`.

La función `opendir()` abre un directorio para poder trabajar con él. Devuelve un manejador de directorio, que, después, mediante la función `readdir()` y un bucle, puede recorrerse para ir leyendo el contenido. Finalmente, con `closedir()` se cierra el manejador de directorio.

A continuación, podemos ver un ejemplo en el que se muestran los ficheros contenidos en una carpeta determinada.

```
<?php
$carpeta="C:/xampp/htdocs/ud04";
$manejador=opendir($carpeta);
if($manejador) {
    $fichero=readdir($manejador);
    while($fichero!=FALSE) {
        if(is_file($fichero)) {
            echo $fichero."<br/>";
        }
        $fichero=readdir($manejador);
    }
}
closedir($manejador);
?>
```

Ejemplo06.php

Como vemos, se crea un manejador para poder movernos por el directorio, y con **readdir** vamos leyendo el contenido. Usamos la función **is_file()**, porque en este caso solo queremos mostrar ficheros, y no directorios. Cuando se llega al final del directorio, la función **readdir** devuelve **false**, salimos del bucle y cerramos el manejador.

Otra forma incluso más sencilla de leer el contenido de un directorio es usar la función **scandir**, que directamente nos deja la lista de archivos del directorio en un *array* que luego podemos procesar de la forma que creamos más conveniente.

```
<?php
$carpeta="C:/xampp/htdocs/ud04";
$lista=scandir($carpeta);
foreach($lista as $fichero) {
    if(is_file($fichero)) {
        echo $fichero."<br/>";
    }
}
?>
```

Ejemplo07.php

En ambos casos, como puedes comprobar, se obtiene por pantalla una lista con los ficheros contenidos en el directorio especificado. Por cierto, si pruebas estos ejemplos en tu ordenador, recuerda cambiar el nombre del directorio para que se corresponda con uno que exista en el servidor sobre el que estés trabajando.

Hay otras posibilidades en PHP, como la función **dir()** o la clase **FilesystemIterator**, que realizan una tarea similar, pero aprovechando la funcionalidad de la programación orientada a objetos. En los enlaces del material complementario puedes encontrar más información sobre estas y otras funciones relacionadas con la gestión del sistema de archivos.

1 CASO PRÁCTICO



ANÁLISIS DE TIPOS DE FICHERO EN UN DIRECTORIO

Desarrolla una pequeña aplicación que muestre cuántos archivos de cada tipo (extensión) existen en un directorio. Para ello, definiremos en una variable el directorio sobre el que queremos trabajar, y posteriormente analizaremos el contenido del directorio especificado y mostraremos un listado en el que aparezcan las extensiones encontradas y la cantidad de archivos que hay de cada extensión. Se nos pide también que mostremos la cantidad de directorios que hay dentro del directorio especificado, sin incluir «.» ni «..», que, como sabes, hacen referencia al propio directorio actual y al superior.

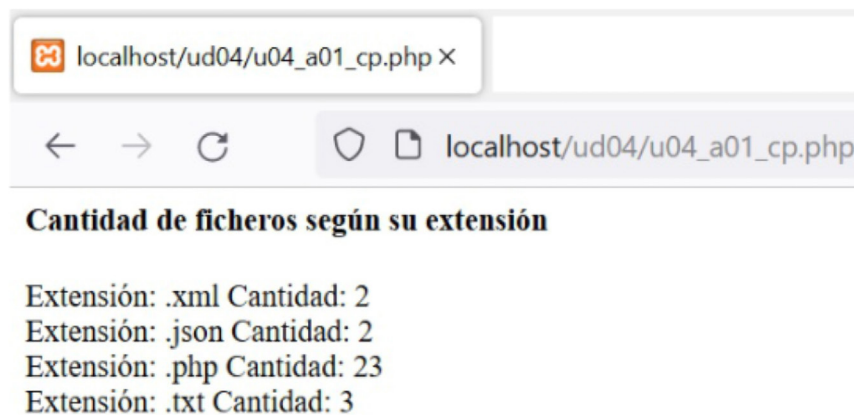
SOLUCIÓN

En estos casos, la mejor forma de implementar un sistema de contadores asociado a unas claves es con *arrays* asociativos. Las extensiones (incluida la palabra «dir» para los directorios) serán las claves, y usaremos el valor de cada clave para contar cuántas veces aparece cada extensión en el directorio escogido. Nos será útil recordar que la función `strrchr()` devuelve la parte de una cadena desde la última ocurrencia de un carácter específico hasta el final, ya que de esa forma podemos obtener la extensión de cada fichero.

A continuación, tienes el código completo de la solución al caso práctico.

```
<?php
$carpeta="C:/xampp/htdocs/ud04";
$extensiones=array(); // definimos un array vacío
$manejador=opendir($carpeta); // abrimos el directorio
// si existe el directorio especificado
if($manejador) {
    // leemos el contenido del directorio
    $fichero=readdir($manejador);
    // hasta que lleguemos al final
    while($fichero!=FALSE) {
        // si es un directorio y no es . ni ..
        if(is_dir($fichero) && $fichero!="." && $fichero!="..") {
            if(array_key_exists("dir",$extensiones)) {
                $extensiones["dir"]++;
            }
            else { $extensiones["dir"]=1; }
        }
        // si es un fichero
        elseif(is_file($fichero)) {
            // obtenemos la extensión
            $extension=strrchr($fichero,".");
            if(array_key_exists($extension,$extensiones)) {
                $extensiones[$extension]++;
            }
            else { $extensiones[$extension]=1; }
        }
        $fichero=readdir($manejador);
    }
    // mostramos la lista
    echo "<h4>Cantidad de ficheros según su extensión</h4>";
    foreach($extensiones as $extension=>$cantidad) {
        echo "Extensión: ".$extension." Cantidad: ".$cantidad."<br/>";
    }
}
closedir($manejador);
?>
```

Puedes ver la solución en el archivo **DWES_U04_CP_A01_SOL.zip**.



02 Operaciones con ficheros de texto no estructurados

PHP ofrece funciones para trabajar con ficheros de texto plano que es interesante conocer, porque estos ficheros, tanto los no estructurados (que vemos en este apartado) como los estructurados (que abordaremos en el siguiente), permiten guardar pequeñas cantidades de información y compartir datos con otras aplicaciones. Vamos a ver a continuación cómo podemos realizar las operaciones básicas (crear, abrir, leer, escribir) sobre ficheros de texto plano.

EJEMPLO

Puedes encontrar todos los ficheros con los ejemplos descargando el archivo **DWES_U04_TEO_A02.zip**, en la sección Ejemplos para trabajar y analizar.

Puedes encontrar todos los ficheros con los ejemplos descargando el archivo **DWES_U04_TEO_A02.zip**, en la sección Ejemplos para trabajar y analizar.

Para trabajar con un fichero de texto en general, necesitamos un manejador para realizar las funciones pertinentes. Para crear ese manejador, disponemos de la función **fopen**. Su sintaxis es:

fopen(nombre_fichero, modo)

La función devuelve un puntero al fichero, que posteriormente usamos para realizar las operaciones. Primero, sin embargo, debemos conocer los diferentes modos de apertura del fichero. El parámetro **modo** indica qué operaciones podemos realizar sobre el fichero, qué ocurre si el fichero no existe, y en qué situación queda el puntero respecto al fichero. Puede tomar los siguientes valores:

MODO	SIGNIFICADO
r	Solo lectura (read). Si el fichero existe, el puntero queda emplazado al principio de este. Si el fichero no existe, se producirá un aviso.
w	Solo escritura (write). El puntero queda situado siempre al principio del fichero, con lo que, si ya existe, se vaciará de contenido. Si el fichero no existe, se creará.
a	Solo escritura (append). El puntero queda situado al final del fichero, con lo que el nuevo contenido se añadirá al final sin sobrescribir el resto. Si el fichero no existe, se creará.
x	Crea un fichero específicamente para escritura, y se produce un aviso si el fichero ya existe. El puntero queda situado al principio del fichero.
c	Solo escritura (create). Si el fichero no existe, se crea. El puntero queda situado al principio del fichero. Si el fichero ya existe, no se producen avisos ni errores, ni tampoco se vacía de contenido, pero al añadir información se sobrescribirá la existente.
r+	Igual que el modo r , pero permite también escritura.
w+	Igual que el modo w , pero permite también lectura.
a+	Igual que el modo a , pero permite también lectura.
x+	Igual que el modo x , pero permite también lectura.
c+	Igual que el modo c , pero permite también lectura.

Tabla 1.

En general, usaremos «**r**» para leer un fichero que ya existe, «**w**» para crear un nuevo fichero para escribir, y «**a**» para añadir datos a un fichero ya existente. Los modos extendidos, básicamente **r+**, **w+** y **a+**, los usaremos cuando queremos realizar operaciones tanto de lectura como de escritura. El modo que escojamos debe responder al tipo de operaciones que queremos realizar.

Un ejemplo típico puede verse en el siguiente código, que abre un fichero para empezar a escribir sobre él. Para evitar perder datos, comprobamos primero si el fichero ya existe, y solo entonces decidimos si lo abrimos con **w** (escribir desde el principio) o con **a** (añadir contenido al final).

```

if(!file_exists("ejemplo.txt")) {
    $f = fopen("samplefile.txt","w"); // se crea el fichero vacío
}
else {
    $f = fopen("samplefile.txt","a"); // se abre para añadir contenido
}

```

2.1. Leer todo el contenido de un fichero

Supongamos que ya tenemos un fichero y queremos leer su contenido. Vamos a ver, de momento, diversas funciones de PHP que permiten leerlo todo de una vez.

readfile (nombre_fichero)

Utilidad: Lee el contenido del fichero y lo manda al búfer de salida (en general, a la pantalla).

file_get_contents (nombre_fichero, inicio, longitud)

Utilidad: Lee el contenido del fichero y lo guarda en una cadena. Permite opcionalmente definir desde dónde empezar a leer (**inicio**) y la cantidad de caracteres que queremos leer (**longitud**; por defecto, hasta el final del fichero).

file (nombre_fichero)

Utilidad: Lee el contenido del fichero y lo transfiere a un *array*, donde cada línea ocupará una posición.

Esquema 6.

Dado el fichero `ejemplo01.txt`, que puedes encontrar dentro del archivo `DWES_U04_TEO_A02.zip`, veamos cómo se lee su contenido con las tres funciones anteriores:

```

<?php
$fichero="ejemplo01.txt";
echo "<h4>Fichero leído con readfile():</h4>";
readfile($fichero);
echo "<h4>Fichero leído con file_get_contents():</h4>";
$cadena=file_get_contents($fichero);
echo $cadena;
echo "<h4>Fichero leído con file():</h4>";
$array=file($fichero);
foreach($array as $linea) { echo $linea."<br/>"; }
?>

```

Ejemplo08.php

El resultado por pantalla sería:

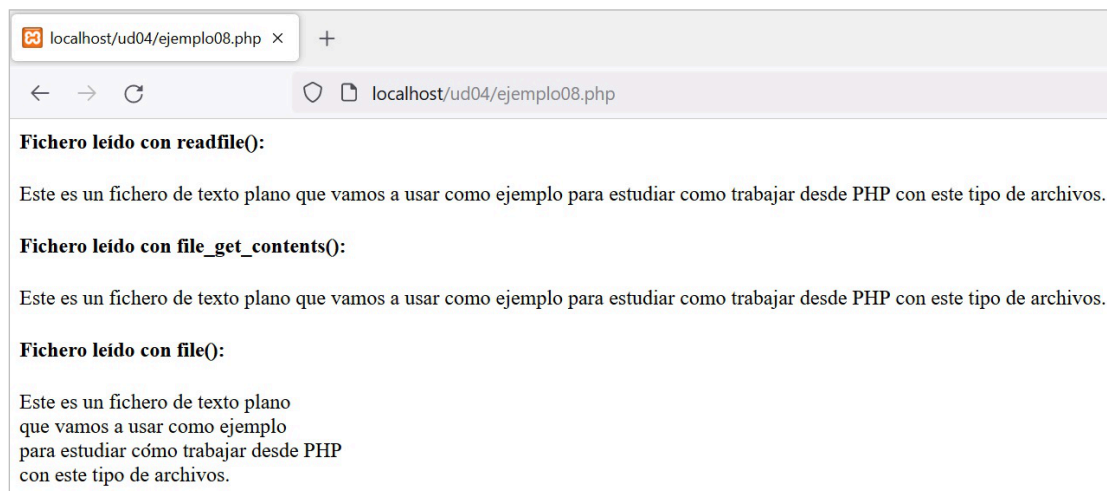


Figura 4.

La mejor opción para trabajar con cada línea del fichero por separado es `file()`. Al devolver un *array* donde cada línea ocupa una posición, nos permite manejarlas de manera diferenciada.

IMPORTANTE

`echo nl2br(file_get_contents("ejemplo01.txt"))`; mostraría cada línea del fichero en una línea distinta por pantalla, al convertir los saltos de línea en etiquetas `<br/>` de HTML.

2.2. Leer el contenido de un fichero por partes

En ocasiones necesitaremos leer un fichero por partes, bien por su excesivo tamaño o porque nos interesa trabajar con cada línea, palabra o carácter por separado. Para ello, necesitamos leerlo, por lo que, en primer lugar, deberíamos abrirlo, por ejemplo, con el modo `r`. Una vez abierto, disponemos de las siguientes funciones:

FUNCIÓN	UTILIDAD
<code>fread (manejador, tamaño)</code>	Lee la cantidad de bytes especificada por tamaño del fichero referenciado por manejador .
<code>fgets (manejador)</code>	Lee una línea del fichero referenciado por manejador y desplaza el puntero a la línea siguiente.
<code>fgetc (manejador)</code>	Lee un carácter del fichero referenciado por manejador y desplaza el puntero al carácter siguiente.
<code>fgetcsv (manejador, longitud, delimitador, cierre)</code>	Lee los bytes especificados por longitud de un fichero con formato CSV; delimitador es el separador de datos (la coma, por defecto), y cierre , el carácter usado para envolver cada dato (comillas, por defecto).

Tabla 2.

Estas funciones leen el fichero por partes, por lo que, si queremos leerlo todo, necesitaremos crear un bucle que acabe cuando lleguemos al final del fichero. La función **feof (manejador)** nos avisa cuando llegamos al final. Así, para leer el fichero ejemplo01.txt hasta el final, podemos hacer:

Línea a línea	Carácter a carácter
<pre> echo "<h4>Línea a línea</h4>"; \$f=fopen(\$fichero,"r"); \$linea=fgets(\$f); while (!feof(\$f)) { echo nl2br(\$linea); \$linea=fgets(\$f); } fclose(\$f); </pre>	<pre> echo "<h4>Carácter a carácter</h4>"; \$f=fopen(\$fichero,"r"); \$caracter=fgetc(\$f); while (!feof(\$f)) { echo "\$caracter
"; \$caracter=fgetc(\$f); } fclose(\$f); </pre>

Esquema 6.

Encontrarás ambas opciones en el fichero **ejemplo09.php**. El resultado de ejecutar el *script* es:

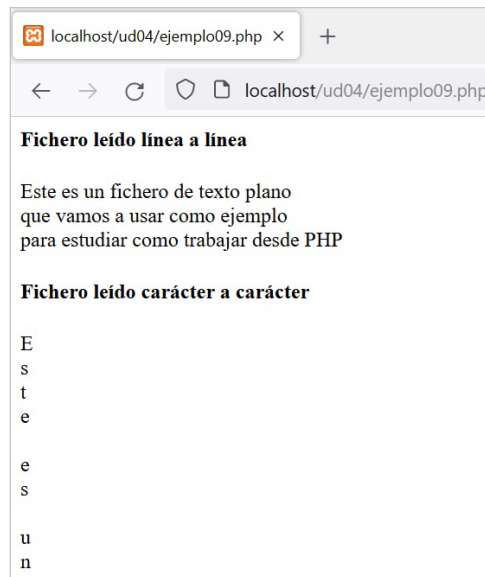


Figura 5.

Vamos a leer un archivo con formato CSV. Puedes descargar el fichero **ejemplo02.txt** y ejecutar el script **ejemplo10.php** (también en la zona de descargas). El contenido de **ejemplo02.txt** es:

```
"Jose","Sánchez","jose@gmail.com","11111111"
"Mercedes","Sanz","merxe@gmail.com","22222222"
"David","Garcia","dgarcia@gmail.com","33333333"
"Amparo","López","alopez@gmail.com","44444444"
```

En este caso, el delimitador y el cierre son los caracteres por defecto, así que no necesitaremos especificarlos. Como vamos a leer línea a línea, tampoco hace falta indicar la longitud en bytes.

```
<?php
$fichero="ejemplo02.txt";
if(file_exists($fichero)) {
    echo "<h4>Fichero CSV</h4>";
    $f=fopen($fichero,"r");
    $array=fgetcsv($f);
    while (!feof($f)) {
        echo "Nombre: ".$array[0]." ".$array[1]."<br/>";
        echo "Email: ".$array[2]."<br/>";
        echo "Teléfono: ".$array[3]."<br/>";
        echo "-----<br/>";
        $array=fgetcsv($f);
    }
    fclose($f);
}
else { echo "El fichero especificado no existe"; }
?>
```

El resultado por pantalla:

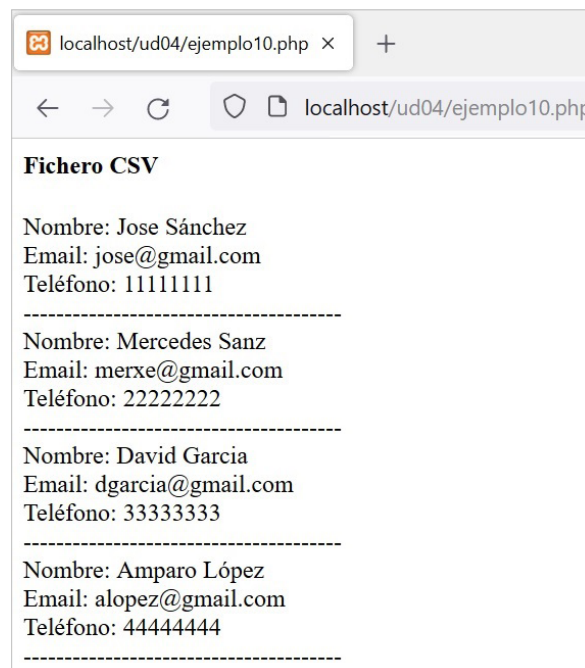


Figura 6.

2.3. Escribir en un fichero

Para escribir en un fichero de texto, debemos abrirlo para que permita la escritura. Si no existe, podemos crearlo con `fopen(nombre_fichero,"w")`, usando el modo `a` para añadir contenido a un fichero existente. En cualquier caso, las funciones `fwrite` y `fputs`, sinónimas, permiten escribir sobre el fichero. En el siguiente ejemplo creamos un fichero, escribimos un par de líneas, lo cerramos, lo volvemos a abrir sin perder su contenido y añadimos dos líneas más.

```
<?php
$fichero="ejemplo03.txt";
$f=fopen($fichero,"w");
fwrite($f,"Línea número 01".PHP_EOL);
fwrite($f,"Línea número 02".PHP_EOL);
fclose($f);
$f=fopen($fichero,"a");
fwrite($f,"Línea número 03".PHP_EOL);
fwrite($f,"Línea número 04".PHP_EOL);
fclose($f);
?>
```

Ejemplo11.php

Al ejecutar este código, se creará un fichero **ejemplo03.txt** con las cuatro líneas especificadas. La constante **PHP_EOL** se añade para que se cree un salto de línea con cada escritura.

La función **fflush(manejador)** se puede usar para forzar la escritura en disco del fichero antes de cerrar el manejador, aunque normalmente no será necesaria. La función **flock(manejador, tipo_bloqueo)** permite bloquear un fichero mientras estamos trabajando con él. Por ejemplo, **flock(\$f, LOCK_EX)** bloquearía el fichero apuntado por **\$f** para escribir sobre él de modo exclusivo. Con **flock(\$f, LOCK_UN)** levantamos el bloqueo.

2.4. Leer y escribir en un fichero

Para poder leer y escribir en un fichero, debemos abrirlo con alguno de los modos que permiten ambas operaciones. Además, necesitaremos mover el puntero para decidir dónde leer y dónde escribir cada vez. Para ello, disponemos de dos útiles funciones:

<code>rewind(manejador)</code>	Rebobina el puntero hasta el inicio del fichero. Así, podemos leer desde el principio, después de haber añadido contenido.
<code>fseek(manejador, bytes, inicio)</code>	El puntero se desplaza la cantidad indicada de bytes desde el punto especificado como inicio. Este punto puede ser el principio del fichero (por defecto; no hace falta especificarlo), la posición actual (SEEK_CUR) o el final del fichero (SEEK_END).

En el ejemplo, añadimos una línea al final del archivo y volvemos al principio para leer su contenido.

```
<?php
$fichero="ejemplo03.txt";
$f=fopen($fichero,"a+");
fwrite($f,"Línea número 05".PHP_EOL);
rewind($f);
echo fgets($f)."<br/>";
while (!feof($f)) { echo fgets($f)."<br/>"; }
fclose($f);
?>
```

Ejemplo12.php

2 CASO PRÁCTICO



DIVIDIMOS UN FICHERO EN DOS

En este caso práctico, vamos a leer el contenido de un fichero existente y, a partir de este, lo volcaremos en otros dos ficheros que deberemos crear en la aplicación. En uno de ellos guardaremos las líneas del fichero original que tengan 25 o más caracteres, mientras que en el otro fichero guardaremos las líneas originales con menos de 25 caracteres. Cada línea del fichero original debe ocupar una línea en los ficheros de destino. El fichero original lo puedes crear con un editor de textos; trata de que tenga al menos unas 20 líneas y de que más o menos la mitad tengan más de 25 caracteres, y la otra mitad, menos. Puedes copiar y pegar, por ejemplo, alguna poesía que te guste que encuentres por Internet o incluso de tu propia invención.

SOLUCIÓN

La solución más sencilla es cargar el fichero original en un *array*, y abrir los otros dos ficheros para escritura en modo «w», de forma que se creen cada vez que ejecutemos el *script*. Luego iremos repasando el *array* posición a posición (línea a línea) y, en función de su longitud, haremos un **fwrite** en uno u otro fichero. Las líneas en blanco, si las hay, no las tendremos en cuenta.

El código fuente completo de la solución a este caso práctico puedes verlo a continuación.

```

<?php
// definimos los ficheros
$original="original.txt";
$largas="largas.txt";
$cortas="cortas.txt";
// abrimos los nuevos ficheros para escritura
// si no existen, se crean
// si existen, se dejan vacios
$fLargas=fopen($largas,"w");
$fCortas=fopen($cortas,"w");
// si existe el fichero original
if(file_exists($original)) {
    // cargamos el fichero en un array
    $versos=file($original);
    // para cada línea del array
    foreach($versos as $verso) {
        // si su longitud es mayor o igual que 25
        if(strlen($verso)>=25) {
            // escribimos la línea en el fichero largas
            fwrite($fLargas,$verso);
        }
        // si es menor que 25 pero mayor que 0
        elseif(strlen($verso) > 0) {
            // lo escribimos en el fichero cortas
            fwrite($fCortas,$verso);
        }
    }
    // forzamos la escritura por si queda algo en el búfer
    fflush($fLargas);
    fflush($fCortas);
    // y cerramos los ficheros
    fclose($fLargas);
    fclose($fCortas);
    echo "<h4>Ficheros copiados</h4>";
}
else {
    echo "El fichero $original no existe<br/>";
}
?>

```

Puedes ver la solución en el archivo `DWES_U04_CP_A02_SOL.zip`.

03 Operaciones con ficheros de texto estructurados

No es frecuente el uso de ficheros de texto para almacenar datos de forma permanente, como si fuesen una base de datos. Es mucho más habitual, correcto, flexible y potente trabajar con una base de datos real. Sin embargo, existen determinados formatos de ficheros de texto con estructura que se han convertido en estándares a la hora de intercambiar datos entre aplicaciones, o entre partes de una misma aplicación, y también para guardar información sobre la configuración de manera permanente. Los más importantes, que trataremos en este apartado, son XML y JSON. El detalle de los formatos en sí se ve con más profundidad en otros módulos como Lenguaje de Marcas, así que aquí nos limitaremos a estudiar cómo trabajar con ambos tipos de ficheros desde PHP.

Puedes encontrar todos los ficheros con los ejemplos descargando el archivo **DWES_U04_TEO_A03.zip**, en la sección Ejemplos para trabajar y analizar.

3.1. PHP y XML

XML (*extensible markup language*) es un lenguaje de marcado muy similar a HTML, ya que se basa en el anidamiento de etiquetas y propiedades. Una agenda como la que hemos visto en el apartado anterior, en formato XML, podría tener esta apariencia:

```
<agenda>
  <contacto id='1'>
    <nombre>José Sánchez</nombre>
    <email>jose@gmail.com</email>
    <telefono>11111111</telefono>
  </contacto>
  <contacto id='2'>
    <nombre>Mercedes Sanz</nombre>
    <email>merxe@gmail.com</email>
    <telefono>22222222</telefono>
  </contacto>
  <contacto id='3'>
    <nombre>David García</nombre>
    <email>dgarcia@gmail.com</email>
    <telefono>33333333</telefono>
  </contacto>
  <contacto id='4'>
    <nombre>Amparo López</nombre>
    <email>alopez@gmail.com</email>
    <telefono>44444444</telefono>
  </contacto>
</agenda>
```

Agenda.xml

Por supuesto, al tratarse de un fichero de texto, lo podríamos leer con las herramientas que hemos visto en el apartado anterior. Sin embargo, PHP provee diversas utilidades y clases para trabajar específicamente con este tipo de archivos. Entre ellas están la biblioteca **SimpleXML**, las clases **XMLWriter** y **XMLReader**, y la clase más genérica **DOMDocument**. Puedes ampliar información sobre todas ellas siguiendo los enlaces que encontrarás en el material complementario. A continuación, vamos a ver un ejemplo de lectura del fichero usando **SimpleXML**.

La biblioteca SimpleXML proporciona una clase denominada **SimpleXMLElement**, para tratar ficheros **XML** y convertir su contenido en una colección de objetos iterable. Podemos crear un objeto **SimpleXMLElement** si tenemos el contenido XML ya cargado en una variable, o hacerlo directamente desde el fichero con la función `simplexml_load_file`. Posteriormente, podemos recorrer el fichero de la siguiente forma:

```
<?php
$fichero="agenda.xml";
if(file_exists($fichero)) {
    $xml=simplexml_load_file($fichero);
    foreach($xml->contacto as $contacto) {
        echo "Id: ".$contacto['id']."<br/>"; // para leer atributos
        echo "Nombre: ".$contacto->nombre."<br/>"; // para leer subetiquetas
        echo "Email: ".$contacto->email."<br/>";
        echo "Teléfono: ".$contacto->telefono."<br/>";
        echo "-----<br/>";
    }
}
else {
    echo "El fichero no existe";
}
?>
```

Ejemplo13.php

Al ejecutar el *script* obtenemos:

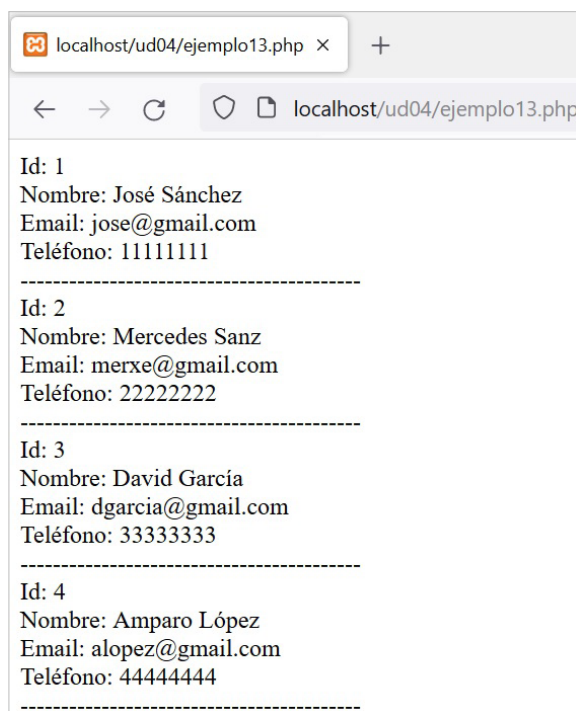


Figura 7.

Como se ve, en caso de que existan atributos dentro de alguna etiqueta, se leen como elementos de un *array* asociativo, mientras que las subetiquetas que están dentro de una etiqueta se leen como propiedades de un objeto.

Para crear un fichero XML también podemos usar **SimpleXML**. El proceso es muy sencillo: en primer lugar, creamos una cadena con la estructura XML vacía; en segundo lugar, la cargamos en un objeto **SimpleXMLElement** mediante **simplexml_load_string**; posteriormente, usamos los métodos **addChild** para añadir elementos y **addAttribute** para añadir atributos a un elemento, y al final usaremos el método **asXML** para guardar el contenido XML que hemos creado en un fichero. A continuación, puedes ver el script completo:

```
<?php
$cadenaXML=<<<<xml

<?xml version='1.0' encoding='UTF-8'?><contactos></contactos>

xml;

$xml=simplexml_load_string($cadenaXML);
$contacto=$xml->addChild('contacto'); // se añade un elemento contacto
$contacto->addAttribute('id','1'); // se crea un atributo para el contacto
$contacto->addChild('nombre','Elena Rodríguez'); // se añaden las
subetiquetas
$contacto->addChild('email','elena@gmail.com');
$contacto->addChild('telefono','55555555');
$contacto=$xml->addChild('contacto'); // un nuevo elemento contacto
```

```

$contacto->addAttribute("id","2");
$contacto->addChild('nombre','Pablo Pérez');
$contacto->addChild('email','pablo@gmail.com');
$contacto->addChild('telefono','66666666');
$xml->asXML("contactos.xml");
?>

```

Ejemplo14.php

Al ejecutar el *script* no se ve nada por pantalla, pero, si usamos un navegador para abrir el fichero **contactos.xml** que hemos creado, podremos ver su contenido.

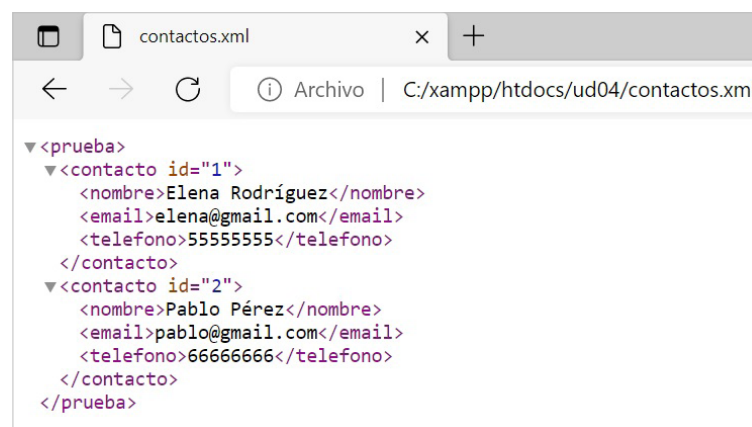


Figura 8.

Por supuesto, siguiendo los pasos que hemos visto hasta ahora, podríamos cargar el fichero en un objeto XML y añadirle nuevos contactos en cualquier momento.

3.2. PHP y JSON

Otro estándar ampliamente usado (de hecho, está prácticamente sustituyendo a XML) para el intercambio de información es JSON. El formato es similar, con parejas **clave-valor** que pueden agruparse en objetos (entre llaves) y *arrays* (entre corchetes). Por ejemplo:

```

{
  "DAW": [
    {
      "modulo": "Programación",
      "curso": 1,
      "horas_semanales": 8
    },
    {
      "modulo": "Bases de datos",
      "curso": 1,
      "horas_semanales": 5
    }
  ]
}

```

```

    },
    {
        "modulo": "Desarrollo web en entorno cliente",
        "curso": 2,
        "horas_semanales": 7
    },
    {
        "modulo": "Desarrollo web en entorno servidor",
        "curso": 2,
        "horas_semanales": 8
    }
]
}

```

Curso.json

La forma más sencilla de leer un fichero en formato JSON es cargar su contenido en una variable con la función `file_get_contents`, que hemos visto en un apartado anterior, y, posteriormente, decodificarlo con la función `json_decode` para convertirlo en un *array* asociativo. Veamos cómo leeríamos el archivo `curso.json`:

```

<?php
$fichero="curso.json";
if(file_exists($fichero)) {
    $json=file_get_contents($fichero);
    $datos=json_decode($json,true);
    foreach($datos["DAW"] as $modulo) {
        foreach($modulo as $clave=>$valor) {
            echo "$clave : $valor <br/>";
        }
        echo "-----<br/>";
    }
}
else { echo "El fichero no existe"; }
?>

```

Ejemplo15.php

Con la función `json_decode` con su segundo parámetro a `true`, el archivo JSON se convierte en un *array* asociativo que, en este caso, solo tiene una posición: «DAW». Dentro de esa posición, el contenido es otro conjunto de *arrays* asociativos que podemos recorrer con bucles `foreach`, tal como vemos en el código fuente del ejemplo. El resultado de ejecutar el *script* sería:

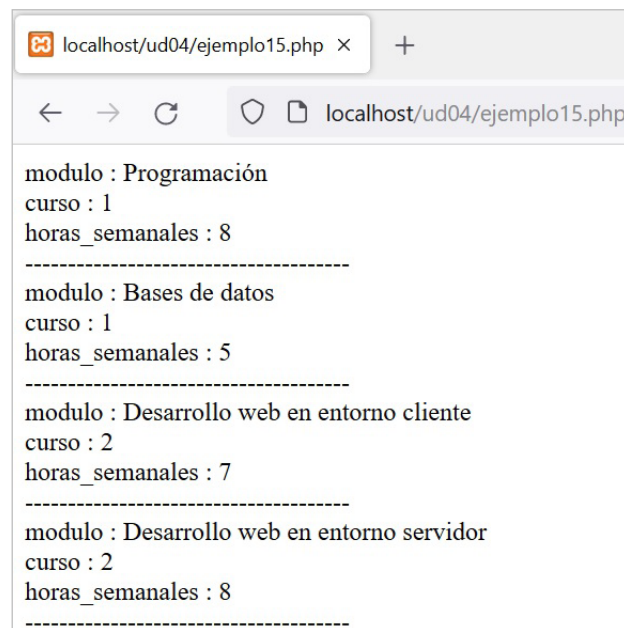


Figura 9.

Una vez que tenemos la información dentro del conjunto de *arrays* asociativos anidados, podemos hacer cualquier cosa que se nos ocurra: listar solo los nombres de los módulos, solo aquellos módulos de primer curso, etcétera.

Si en la función `json_decode` no incluimos el segundo parámetro, entonces en lugar de un *array* asociativo obtenemos un objeto con una propiedad, el nodo raíz, que incluye un *array* de objetos. La lectura es parecida, pero con algunos cambios, como podemos ver a continuación:

```

<?php
$fichero="curso.json";
if(file_exists($fichero)) {
    $json=file_get_contents($fichero);
    $datos=json_decode($json); // ahora $datos es un objeto
    // con una propiedad "DAW" que a su vez contiene un array de objetos
    foreach($datos->DAW as $modulo) {
        echo "Módulo: ".$modulo->modulo."<br/>";
        echo "Curso: ".$modulo->curso."<br/>";
        echo "Horas semanales: ".$modulo->horas_semanales."<br/>";
        echo "-----<br/>";
    }
}
else { echo "El fichero no existe"; }
?>

```

El resultado por pantalla será exactamente el mismo que en la figura anterior.

En cuanto a la creación de estructuras o archivos JSON, es también bastante sencilla. Para crear un JSON usamos la función `json_encode`. Lo más sencillo es partir de un `array` asociativo con los nodos y los pares clave-valor que queramos incluir en la estructura y pasárselo a la función. Por ejemplo:

```
<?php
$datos=array("DAW"=>array(
    array("modulo"=>"Lenguaje de marcas","curso"=>1,"horas"=>3) ,
    array("modulo"=>"Sistemas informáticos","curso"=>1,"horas"=>5) ,
    array("modulo"=>"Despliegue de aplicaciones web","curso"=>2,"horas"=>4)
));
$json=json_encode($datos);
$bytes=file_put_contents("modulos.json",$json);
?>
```

Ejemplo17.php

Al ejecutar el *script*, se creará el fichero **modulos.json**, que, al abrirlo con el navegador, se verá así:

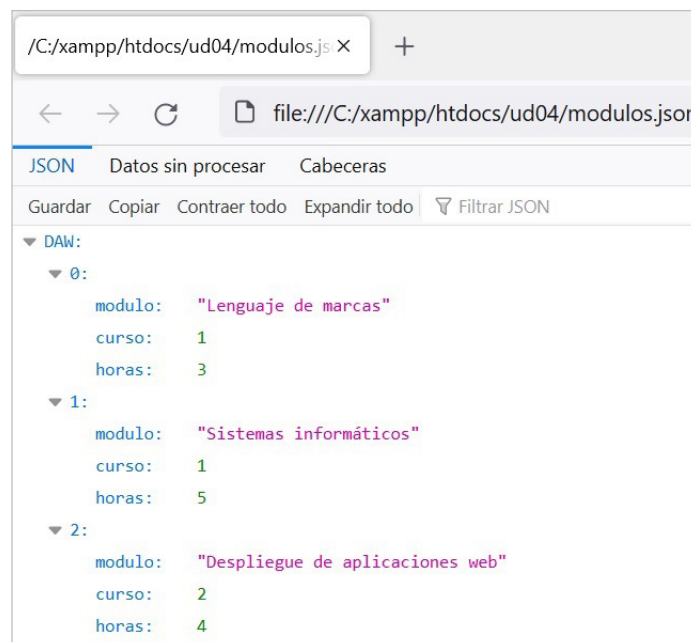


Figura 10.

04 Operaciones con bases de datos relacionales

4.1. Conexión a la base de datos

Lo primero que hay que hacer para trabajar con una base de datos es establecer una conexión. En nuestro caso, creamos un objeto PDO con el nombre del host, el de la base de datos, el del usuario y la contraseña.

```
new PDO('mysql:host=hostname;dbname=database', 'usuario', 'password')
```

Esta instrucción puede fallar si la base de datos no existe, el host no está disponible, el usuario no existe o la contraseña es incorrecta, por ejemplo. Para gestionar los errores, usamos la excepción

```
PDOException:
```

```
try {  
    $pdo = new PDO('mysql:host=localhost;dbname=prueba', 'usuario', '1234');  
    echo "Conexión establecida";  
}  
catch (PDOException $e) { echo "Error en la conexión"; }
```

El constructor de la clase PDO siempre lanzará una excepción si falla la conexión. Si vemos que, por defecto, otros fallos de PDO no lanzan excepciones, modificamos la configuración de la gestión de errores cambiando un atributo del objeto para que convierta los errores en excepciones:

```
$pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
```

Otro ajuste importante que tal vez necesitemos es el del juego de caracteres; básicamente, para que coincida con el usado en la base de datos. Así, si en nuestra base de datos trabajamos con UTF-8 para poder usar caracteres como eñes, ciertas vocales con tilde, etcétera, en el objeto creado por la conexión haremos:

```
$pdo->exec('SET NAMES "utf8"');
```

Aunque nos conectemos habitualmente a la base de datos como usuario root para tareas de administración, no es conveniente hacerlo desde la aplicación. Es mejor práctica crear un usuario (o varios) con los permisos básicos que necesitemos para poder usar la aplicación y hacer la conexión con dicho usuario.

4.2. Altas, bajas, modificaciones y consultas (CRUD)

Para lanzar instrucciones SQL, que básicamente en una aplicación web de usuario serán consultas, inserciones, borrados y modificaciones, disponemos de dos métodos de la clase PDO:

MÉTODO	USO
<code>\$pdo->exec(\$sql)</code>	Para operaciones INSERT , DELETE y UPDATE . Le pasamos la cadena <code>\$sql</code> con la instrucción, y devuelve la cantidad de filas afectadas.
<code>\$pdo->query(\$sql)</code>	Para SELECT o cualquier operación que retorne datos. Devuelve una lista con las filas de la tabla o tablas que han sido seleccionadas por la consulta.

Tabla 3.

Veamos unos ejemplos. Supongamos que disponemos de un SGBD MySQL/MariaDB y una base de datos **mcgraw_agenda** con una tabla agenda (puedes encontrar en **DWES_U04_TEO_A04.zip** un *script* SQL llamado **mcgraw_agenda**, que te permitirá crear la base de datos y la tabla vacía) con la siguiente estructura:

Estructura de tabla		Vista de relaciones					
#	Nombre	Tipo	Cotejamiento	Atributos	Nulo	Predeterminado	Extra
☐ 1	idContacto 	int(11)			No	Ninguna	AUTO_INCREMENT
☐ 2	nombreContacto	varchar(20)	utf8_spanish2_ci		No	Ninguna	
☐ 3	apellidosContacto	varchar(30)	utf8_spanish2_ci		No	Ninguna	
☐ 4	emailContacto	varchar(30)	utf8_spanish2_ci		No	Ninguna	
☐ 5	tfnoContacto	varchar(10)	utf8_spanish2_ci		No	Ninguna	

Vamos a conectar con la base de datos (deberás poner tu nombre de usuario y tu contraseña):

```
// conexión
try {
    $pdo = new PDO('mysql:host=localhost;dbname=mcgraw_agenda','fidel',
'1234');

    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
    $pdo->exec('SET NAMES "utf8"');
    echo '<h4>Conexión establecida</h4>';
}
catch (PDOException $e) {
    echo 'Error en la conexión: ' . $e->getMessage();
}
```

El mensaje «**Conexión establecida**» indica que ha ido todo bien y estamos conectados. En caso de error, deberemos comprobar la cadena con la que hemos creado el objeto PDO. Ahora insertamos un nuevo contacto:

```
// inserción de un nuevo contacto
try {
    $sql="INSERT INTO agenda VALUES(NULL,'José','Sánchez','jose@gmail.com','11111111')";
    $filasInsertadas=$pdo->exec($sql);
    echo "Se han añadido $filasInsertadas filas<br/>";
}
catch(PDOException $excepcion) {
    echo "Error en la inserción de tipo ".$excepcion->getMessage();
}
```

Como antes, si vemos el mensaje «**Se han añadido 1 filas**» es que ha ido todo bien. Si se produce un error en la inserción a pesar de habernos conectado correctamente, hay que revisar la cadena SQL.

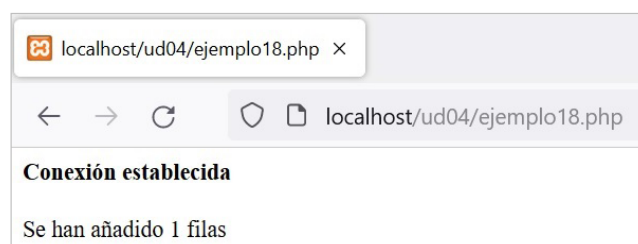


Figura 12.

Ahora listamos los contactos para verificar el alta. Para ello, usamos el método `query()` pasando como parámetro la consulta SQL deseada. Se devuelve un objeto con varios *arrays* asociativos, cuyas claves son los nombres de las columnas en la tabla, y cuyos valores son su contenido. El método `fetch()` permite saltar a la fila siguiente para recorrer la estructura y mostrar los datos. Al llegar al final de la lista devuelve `null`, y entonces acabaremos el bucle. Veamos cómo quedaría la consulta y el listado de los contactos:

```
try {
    $sql="SELECT * FROM agenda";
    $lista=$pdo->query($sql);
    echo "<h4>Lista de contactos</h4>";
    while($contacto = $lista->fetch()) {
        echo "Nombre: ".$contacto['nombreContacto']. "
        ".$contacto['apellidosContacto'];
        echo " Email: ".$contacto['emailContacto'];
        echo " Teléfono: ".$contacto['tfnoContacto'];
```



```

    }
}
catch(PDOException $excepcion) {
    echo "Error en la consulta de tipo ".$excepcion->getMessage();
}

```

Si muestra un error, revisaremos la cadena SQL que hemos utilizado. En caso contrario, veremos la lista:

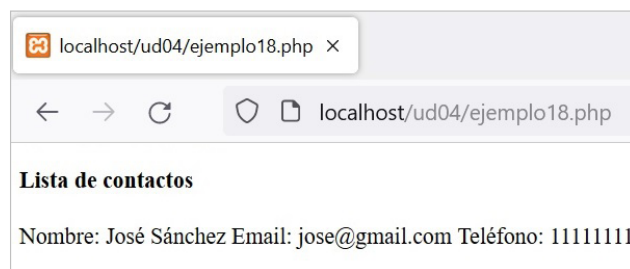


Figura 13.

Las modificaciones (**UPDATE**) y los borrados (**DELETE**) se realizan igual que las inserciones (**INSERT**). El siguiente código cambia el correo del contacto que acabamos de insertar.

```

// hacemos una modificación en los datos del contacto
try {
    $sql="UPDATE agenda SET emailContacto='jsanchez@gmail.com' WHERE
idContacto=1";
    $filasModificadas=$pdo->exec($sql);
    echo "Se han modificado $filasModificadas filas<br/>";
}
catch(PDOException $excepcion) {
    echo "Error en la modificación de tipo ".$excepcion->getMessage();
}

```

Si volvemos a mostrar los contactos, comprobaremos que el cambio se ha realizado.

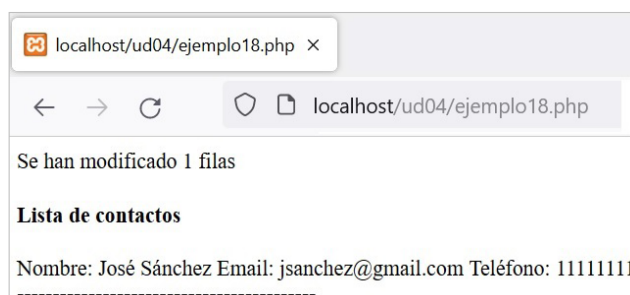


Figura 14.

El borrado del contacto es similar; solo cambia la instrucción SQL.

```
// borramos el contacto
try {
    $sql="DELETE FROM agenda WHERE idContacto=1";
    $filasBorradas=$pdo->exec($sql);
    echo "Se han borrado $filasBorradas filas<br/>";
}
catch(PDOException $excepcion) {
    echo "Error en el borrado de tipo ".$excepcion->getMessage();
}
```

Si ahora volvemos a listar los contactos, veremos que la lista está vacía. Dispones del ejemplo completo en el fichero **ejemplo18.php** (dentro del archivo **DWES_U04_TEO_A04.zip**, en el apartado Ejemplos para trabajar y analizar).

EJEMPLO

En el apartado Descarga de documentos encontrarás una amplia explicación de un ejemplo similar en el archivo **PHP y Bases de Datos usando MySQLi**. El código completo lo encontrarás en el fichero **DWES_U04_MC_E.zip** (en el apartado Ejemplos para trabajar y analizar).

4.3. Seguridad

Al operar con bases de datos, hay que evitar las inyecciones de código SQL malicioso. Para ello, además de sanear la información de entrada como ya hemos visto, podemos usar **declaraciones preparadas**. En una declaración preparada no se incluyen los datos directamente en la cadena SQL, sino que se definen unas variables a las que daremos valor posteriormente. Veamos un ejemplo: supongamos que vamos a insertar un contacto cuyos datos nos llegan desde un formulario a través de `$_POST`. Esto es lo que no debemos hacer:

```
$sql = "INSERT INTO agenda VALUES (NULL, $_POST['nombre'] ... )"
```

Es mucho mejor usar una declaración preparada. Primero preparamos la sentencia con los parámetros:

```
$sql = 'INSERT INTO agenda
VALUES (null, :nombre, :apellidos, :email, :tfno) ' ;
$stmt = $pdo->prepare($sql);
```

Ahora una vez extraídos los datos de `$_POST`, debidamente saneados, se los asignamos a los parámetros, al tiempo que ejecutamos la sentencia:

```
$stmt->execute(array( ':nombre'=> strip_tags($_POST['nombre']),
':apellidos'=> strip_tags($_POST['apellidos']),
':email'=> strip_tags($_POST['email']),
':telefono'=> strip_tags($_POST['telefono'])));
```

Las declaraciones preparadas pueden usarse también para realizar consultas. En ese caso, podemos pasar el resultado devuelto por la consulta a un *array* con el método `fetchAll()`.

```
$resultados=$stmt->fetchAll();
```

También se usan signos de interrogación en lugar de nombres de variables. Por ejemplo, `$sql = 'INSERT INTO agenda VALUES (null,?,?,?,?,?)'`. En este caso, para asignar valores a los parámetros basta con ponerlos en el orden adecuado:

```
$stmt->execute(array($nombre, $apellidos, $email, $telefono));
```

Otro aspecto importante de la seguridad es el almacenamiento en la base de datos de las contraseñas de los usuarios. PHP permite encriptarlas antes de guardarlas, con la función `password_hash`:

```
$passOriginal="1234" // la contraseña
$passCodificado = password_hash($passOriginal, PASSWORD_DEFAULT);
$sql = "INSERT INTO tabla(usu, pass) VALUES ('$name', '$passCodificado')";
```

Si entramos directamente a la base de datos como administradores, veremos que la contraseña está encriptada. Al leerla con una consulta, la compararemos con la contraseña original con

```
password_verify.
if (password_verify($passOriginal, $passCodificado))...
```

4.4. Transacciones

En las bases de datos existe el concepto de **transacción**. Una transacción engloba una serie de operaciones que deben ejecutarse conjuntamente, de forma que, si una falla, se anulan todas. En PDO, cada operación genera su propia transacción. Si queremos agrupar instrucciones en una transacción, debemos hacerlo manualmente.

```
<?
```

```
try {
$stmt->beginTransaction(); // empieza la transacción
$stmt->exec($sql1);         // se ejecuta la primera instrucción del bloque
$stmt->exec($sql2);         // se ejecuta la segunda instrucción del bloque
$stmt->exec($sql3);         // se ejecuta la tercera instrucción del bloque
$stmt->commit();            // confirma los cambios
```

```
} catch (Exception $e) {  
    $pdo->rollBack();          // Si alguna instrucción falla, rollBack deshace  
                                // los cambios y se vuelve a la situación inicial  
    echo "Ha habido un error: " . $e->getMessage();  
}  
?>
```

4.5. El problema de las fechas

En las bases de datos MySQL/MariaDB, las fechas se guardan en formato año-mes-día. Esto no tiene importancia, y además permite ordenar mejor por fecha, pero hemos de tenerlo en cuenta al realizar tanto consultas como inserciones o modificaciones.

En SQL disponemos de la función **DATE_FORMAT**, que podemos usar en las consultas para que devuelva la fecha en el formato deseado.

```
SELECT DATE_FORMAT(fecha, '%D %M %Y') from tabla;
```

En cualquier caso, recordemos que disponemos de numerosas funciones PHP para manejar fechas, establecer formatos, etcétera. Así, podemos convertir una fecha a formato YYYY-MM-DD antes de guardarla en la base de datos.

Otra opción interesante es guardar las fechas directamente en formato **timestamp**, que es el usado por Unix/Linux y representa la cantidad de segundos transcurridos desde que empezó a contar el tiempo Unix. Es un entero, puede guardarse como tal en la base de datos y, al recuperarse desde PHP, se puede usar con la función **date()** para revertirlo al formato que deseemos. Además, **timestamp** tiene la ventaja de que no depende de la zona horaria.

Dada una variable **\$dia** con una fecha, podríamos hacer **\$fecha = \$dia->getTimestamp()** para obtener su **timestamp**, guardarlo en la base de datos y, al recuperarlo de la misma desde PHP, usar la función **date(\$formato, \$timestamp)** para mostrarlo con el formato que creamos conveniente.

05 Operaciones con bases de datos NoSQL

Como hemos comentado, las bases de datos NoSQL están ganando popularidad en los últimos tiempos, sobre todo por su flexibilidad y mejor adaptación a todo tipo de dispositivos. En este apartado estudiaremos cómo trabajar desde PHP con una de las bases de datos no relacionales más populares: **MongoDB**. Lógicamente, no tenemos espacio para repasar todas las posibilidades que ofrece MongoDB, pero al menos veremos cómo realizar las operaciones básicas con una base de datos de este tipo desde una aplicación en PHP.

5.1. Particularidades de MongoDB

MongoDB no es una base de datos relacional, así que su nomenclatura es un tanto diferente. Lo que en las bases de datos SQL denominamos tablas, aquí son **colecciones**. Por su parte, no existen filas ni columnas como tales: el contenido de la colección está formado por **documentos** con su propia estructura.

Para poder trabajar sobre MongoDB desde PHP instalaremos la extensión **MongoDB**, que funciona con versiones de PHP iguales o posteriores a la 5.4. Para ello (en Windows) vamos a la página <https://pecl.php.net/package/mongodb/1.13.0/windows> y descargamos la versión que nos interese, en función de nuestro sistema operativo, y, muy importante, de nuestra versión de PHP. El fichero descargado estará comprimido. Lo descomprimimos, copiamos el archivo **php_mongodb.dll** a la carpeta donde tenemos las extensiones de PHP (en Windows, si trabajamos con XAMPP, será C:\xampp\php\ext), y finalmente añadimos la línea **extension=php_mongodb.dll** al fichero de configuración **php.ini**.

```
930 ;extension=ldap
931 extension=mbstring
932 extension=exif ; Must be
933 extension=mysqli
934 extension=php_mongodb.dll
935 ;extension=oci8_12c ; Use wi
936 ;extension=oci8_19 ; Use wit
```

Figura 15.

Ahora necesitamos instalar un servidor MongoDB en nuestro ordenador para realizar las pruebas y seguir los ejemplos. Desde la página oficial de MongoDB, en la **sección de descargas**, seleccionamos plataforma y versión y descargamos el archivo (un fichero **msi** en Windows). Opcionalmente, podemos hacer lo mismo con **mongosh**, un **shell** para acceder al servidor y navegar por su contenido. No podemos entrar aquí en más detalles sobre la instalación y configuración de MongoDB. En adelante, asumimos que tenemos MongoDB instalado, funcionando, y esperando conexiones a través del puerto 27017.

Una vez en marcha MongoDB, si ejecutamos desde la línea de comandos el programa **mongo** accederemos a un entorno en el que podemos ver las bases de datos y colecciones de que disponemos. Por ejemplo, vamos a listar las bases de datos que hay instaladas por defecto:

```
cmd mongosh mongodb://127.0.0.1:27017/?directConnection=true&
test> show databases
admin    40.00 KiB
config   60.00 KiB
local    72.00 KiB
test>
```

Figura 16.

En MongoDB no hay instrucciones para crear bases de datos: se crean automáticamente en cuanto empezamos a añadir colecciones y documentos dentro de las colecciones. Así pues, pasemos rápidamente a ver cómo realizar las operaciones básicas (altas, bajas, modificaciones y consultas) sobre una base de datos MongoDB.

5.2. Conexiones y consultas básicas sobre MongoDB

En primer lugar, crearemos un usuario con permisos de lectura y escritura sobre la base de datos **test**. Desde la línea de comandos que se nos ha abierto al ejecutar **mongo**, escribimos:

```
db.createUser({user:"fidel",pwd:"1234",roles:[{role:"readWrite",db:"test"}]});
```

Acabamos de crear un usuario «fidel» con contraseña «1234» y permisos de lectura y escritura sobre la base de datos **test**. Ahora podemos intentar la conexión, aunque primero debemos importar una serie de clases que vamos a necesitar. Posteriormente, creamos la conexión con la línea:

```
$conexion = new Manager('mongodb://fidel:1234@localhost:27017/test');
```

Hemos especificado el usuario, la contraseña, el servidor con su puerto, y finalmente el nombre de la base de datos. Ahora vamos a ver el contenido de la colección «agenda» dentro de la base de datos «test»:

```
$contactos = $conexion->executeQuery("test.agenda", new Query([], []));  
var_dump($contactos->toArray());
```

Lógicamente, no nos mostrará nada por pantalla, porque la colección «agenda» no existe todavía, pero tampoco obtendremos ningún error. El código completo hasta ahora sería el siguiente:

```
<?php
```

```
use \MongoDB\Driver\Manager;  
use \MongoDB\Driver\Query;
```

```
// Iniciar conexión
```

```
$conexion = new Manager('mongodb://fidel:1234@localhost:27017/test');
```

```
// Obtener todos los registros de la colección agenda
```

```
$contactos = $conexion->executeQuery("test.agenda", new Query([], []));  
var_dump($contactos->toArray());
```

```
?>
```

5.3. Alta de documentos

Con el *driver* MongoDB que estamos usando, para insertar documentos en una colección necesitamos importar la clase **BulkWrite**. Posteriormente, creamos un objeto de dicha clase, y ejecutamos su método **insert()** con el documento que queremos añadir a la colección. Finalmente, el método **executeBulkWrite()** lanzará la inserción sobre la colección que especifiquemos. Veamos cómo dar de alta un contacto en la agenda:

```
// Damos de alta un contacto
$contacto = new BulkWrite();
$contacto->insert([ 'nombre'=>'Jose' ,
                    'apellidos'=>'González' ,
                    'email'=>'jose@gmail.com' ,
                    'telefono'=>'12121212' ] );
$conexion->executeBulkWrite("test.agenda", $contacto);
```

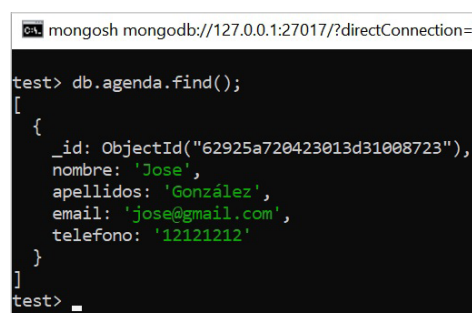
Si ahora volvemos a obtener la lista de contactos y la mostramos con **var_dump**, veremos que nos muestra datos, aunque de una manera bastante ininteligible. De momento, nos conformamos con eso. Si ahora volvemos a nuestro entorno de MongoDB y ejecutamos **use test** (para acceder a la base de datos **test**) y **show collections** (para ver las colecciones que tenemos), podemos ver que nos muestra la colección **agenda**.



```
mongosh mongodb://127.0.0.1:27017/?directConnectio
test> show collections
agenda
test>
```

Figura 17.

Y si ejecutamos el comando **db.agenda.find()**, podremos ver el contenido de la colección:



```
mongosh mongodb://127.0.0.1:27017/?directConnection=
test> db.agenda.find();
[
  {
    _id: ObjectId("62925a720423013d31008723"),
    nombre: 'Jose',
    apellidos: 'González',
    email: 'jose@gmail.com',
    telefono: '12121212'
  }
]
test> _
```

Figura 18.

El código completo del ejemplo lo puedes encontrar en la zona Ejemplos para trabajar y analizar, con el nombre de **ejemplo20.php**, dentro del archivo **DWES_U04_TEO_A05.zip**. Ahora, veamos cómo realizar consultas con filtros y mostrar el resultado por pantalla de forma más atractiva.

5.4. Consulta y visualización de documentos

Recordemos cómo obteníamos todos los documentos:

```
$contactos = $conexion->executeQuery("test.agenda", new Query([], []));
```

Para realizar una consulta con filtros, al crear el objeto `Query` debemos incluir, como primer parámetro, la condición de búsqueda, y como segundo (opcional), indicaciones como el orden, el límite de filas, etcétera. Vamos a añadir tres contactos más a nuestra colección **Agenda**, algo que podemos hacer desde la aplicación o desde el propio entorno de MongoDB con las funciones `insertOne()` o

```
insertMany() :
```

```
test>db.agenda.insertOne({nombre:'Ana',apellidos:'López',email:'  
'ana@gmail.com',telefono:'23232323'});  
test>db.agenda.insertMany([{nombre:'David',apellidos:'Sanz',email:'  
david@yahoo.es',telefono:'3434343434'}, {nombre:'Alberto',apellidos:'  
García',email:'agarcia@hotmail.com',telefono:'55555555'}]);
```

Ahora vamos a listar todos los contactos de una forma más limpia. Volvemos a ejecutar la consulta sin filtros, pero esta vez recorreremos la lista con un bucle `foreach`, que va extrayendo los documentos encontrados en forma de objetos, con lo que podemos acceder a sus propiedades, como a cualquier atributo de un objeto.

```
$contactos = $conexion->executeQuery("test.agenda", new Query([], []));  
foreach($contactos as $contacto) {  
    echo "Nombre: ".$contacto->nombre." ".$contacto->apellidos."<br/>";  
    echo "Email: ".$contacto->email." Teléfono: ".$contacto->  
>telefono."<br/>";  
    echo "<hr>";  
}
```

El resultado será:

Nombre: Jose González Email: jose@gmail.com Teléfono: 12121212
Nombre: Ana López Email: ana@gmail.com Teléfono: 23232323
Nombre: David Sanz Email: david@yahoo.es Teléfono: 3434343434
Nombre: Alberto García Email: agarcia@hotmail.com Teléfono: 55555555

Figura 19.

La misma consulta, ordenando por nombre:

```
$contactos = $conexion->executeQuery("test.agenda",
    new Query([], ['sort'=>array('nombre'=>1)]));
```

La misma consulta, solo con los nombres que empiezan por «A»:

```
$contactos = $conexion->executeQuery("test.agenda",
    new Query(['nombre'=>new \MongoDB\BSON\regex('A')],
    ['sort'=>array('nombre'=>1)]));
```

La misma consulta, con los teléfonos que empiezan con una cifra mayor que 2:

```
$contactos = $conexion->executeQuery("test.agenda",
    new Query(['telefono'=>['$gte'=>'3']], ['sort'=>array('nombre'=>1)]));
```

Las posibilidades de filtro, como puedes ver, son numerosas. Hay que investigar para encontrar la que mejor funciona para la consulta que necesitamos. Puedes encontrar los ejemplos de esta sección en el fichero **ejemplo21.php**.

5.5. Modificación de documentos

Para modificar uno o varios documentos, necesitamos de nuevo la clase **BulkWrite**. Su método **update()** nos permite realizar cambios en los documentos. Para ello, necesitamos definir tres parámetros: la condición de filtro (qué documentos queremos modificar), el cambio que queremos hacer, y si queremos modificar todos los documentos que cumplan la condición. Por ejemplo, vamos a cambiar el teléfono de Ana López de «23232323» a «4545454545».

```
// Cambiar el teléfono de Ana López
// usamos $and en el filtro para definir dos condiciones
// que se han de cumplir a la vez
$filter=[ '$and'=>[['nombre'=>'Ana'], ['apellidos'=>'López']] ];
// definimos el cambio que queremos hacer con $set
$cambio=[ '$set'=>['telefono'=>'4545454545'] ];
// y ahora creamos el BulkWrite
$contacto = new BulkWrite();
// lanzamos el update, la opción multi a true indica que
// se modificarán todos los contactos que cumplan la condición
$contacto->update($filter,$cambio, ['multi'=>true]);
// y lo ejecutamos sobre la colección agenda
$conexion->executeBulkWrite("test.agenda", $contacto);
```

Si volvemos a mostrar la lista de contactos, veremos que el teléfono de Ana López ha cambiado.

El método **update()** todavía admite un nuevo parámetro: **upsert**. Si se pone a **true**, entonces, en el caso de que no haya ningún contacto que cumpla con la condición establecida, se creará uno nuevo.

```
$contacto->update($filter,$cambio, ['multi'=>true,'upsert'=>true]);
```

5.6. Eliminar un documento

Para eliminar un documento, usamos el método `delete()` de la clase `BulkWrite`. En este caso, solo hará falta especificar el filtro. Opcionalmente, podemos añadir el parámetro `limit`, que si está a **false** (por defecto) hará que se eliminen todos los documentos que cumplan la condición del filtro; mientras tanto, si está a **true** solo se eliminará el primer documento encontrado que la cumpla.

Vamos a borrar el contacto con el teléfono «55555555».

```
// Eliminamos el contacto cuyo teléfono es 55555555
$filter=[ 'telefono'=>'55555555' ];
$contacto = new BulkWrite();
// lanzamos el delete, la opción limit a true hace que
// solo se elimine el primer contacto que contenga el teléfono
$contacto->delete($filter,[ 'limit'=>true]);
// y lo ejecutamos sobre la colección agenda
$conexion->executeBulkWrite("test.agenda", $contacto);
```

Si consultamos la lista de contactos, veremos que ha desaparecido el que tenía el teléfono definido en el filtro.

Lista de contactos tras la eliminación del contacto	
Nombre: Jose González	
Email: jose@gmail.com	Teléfono: 12121212
Nombre: David Sanz	
Email: david@yahoo.es	Teléfono: 3434343434
Nombre: Ana López	
Email: ana@gmail.com	Teléfono: 223223223

Figura 20.

Puedes encontrar el ejemplo completo de modificación y eliminación de contactos en el fichero **ejemplo22.php**.

Este ha sido un repaso básico al uso del *driver* MongoDB de PHP para trabajar con bases de datos MongoDB. Existen muchas más funciones, y también otra opción diferente: la **biblioteca MongoDB**, que ofrece otras clases distintas a las del *driver*. Te animamos a profundizar en su uso a partir de los enlaces que podrás encontrar en el documento adjunto



VÍDEOLAB

Para consultar el siguiente vídeo sobre Página Login, escanea el código QR o [pulsa aquí](#).

UAX FP